Frontline

Tech stack and architecture overview

Generated 2026-10-20 from the repository, dev-docs, and project notes. The code is the source of truth. Items marked **verify** come from notes that may be stale.

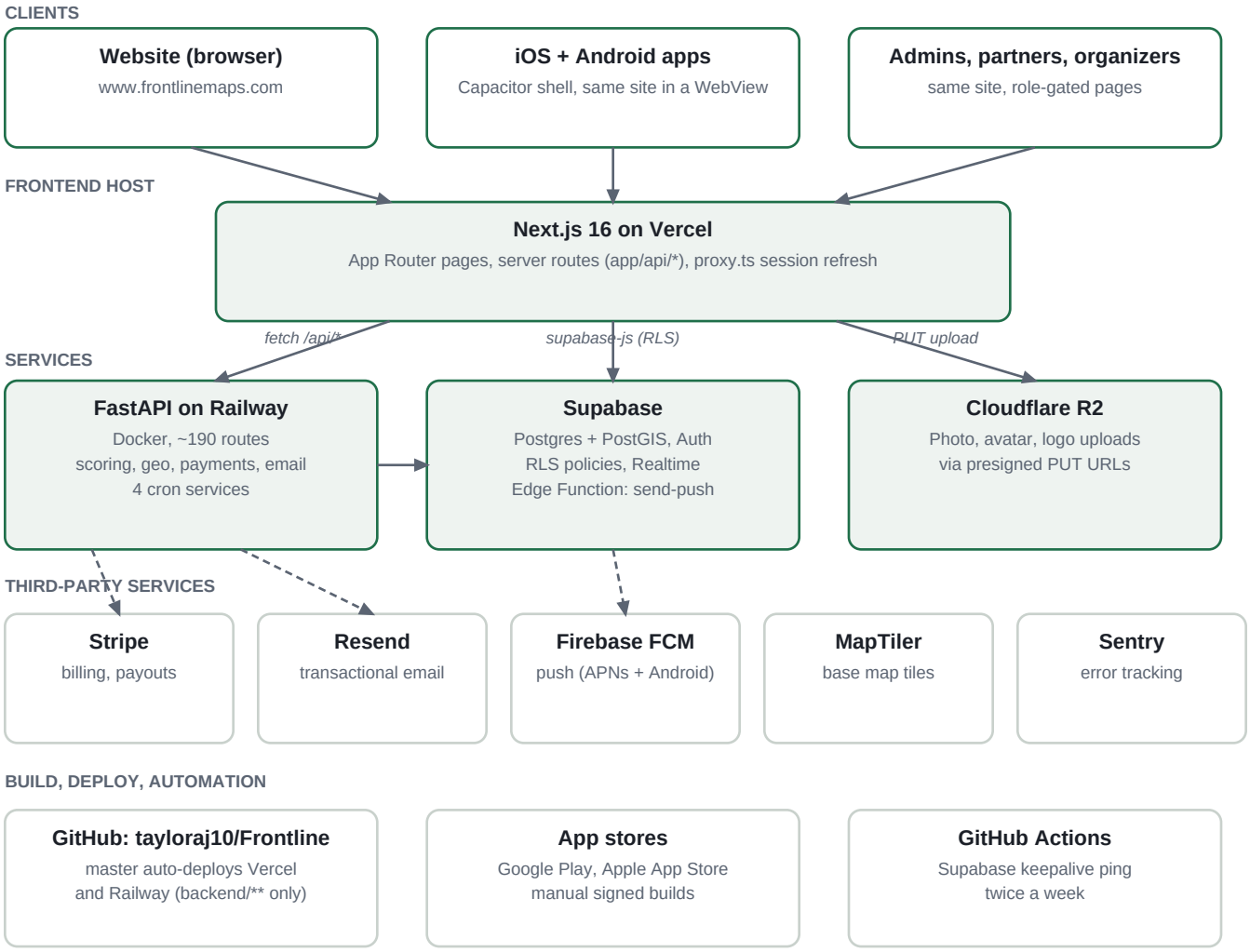
1. What Frontline is

Frontline is a gamified collective-action platform. People join **campaigns** (the flagship is Trash War, a litter cleanup campaign), log real-world **contributions** such as bags or pounds collected with GPS and photos, and see results on a live map and leaderboards. Groups host **cleanup events** with RSVP, GPS or code check-in, and organizer totals. Partner businesses offer rewards redeemable with earned points. A payments layer (Frontline Plus, sponsorships, sponsored cleanups) is built but hidden from regular users behind game settings.

It ships as one product in three forms: a **website**, an **Android app**, and an **iOS app**. The mobile apps are thin native shells that load the live website, so a web deploy updates all three at once.

The app has been live in production since 2026-06-23, so every change has real-user blast radius.

2. System at a glance



Dashed arrows are calls to external vendors. Supabase and FastAPI share one Postgres database: the frontend talks to it directly through Supabase (protected by RLS), and FastAPI talks to the same database over a direct SQL connection (bypassing RLS). The browser also calls MapTiler (map tiles) and Sentry (errors) directly.

3. The deployed product (what users and operators see)

3.1 Surfaces

Surface	Where	Notes
Website	www.frontlinemaps.com (also frontlinemaps.vercel.app, Vercel previews per branch)	Next.js 16 on Vercel. Generated sitemap and robots routes. Terms and Privacy pages.
Android app	Google Play, package com.frontlinemaps.app	Capacitor 8 shell. Notes say a version bump was awaiting Play review (verify current status).
iOS app	Apple App Store, bundle com.frontlinemaps.app, Apple Team ID 4PF46V9GR7	Capacitor 8 shell, built on a Mac with Xcode. Review status: verify in App Store Connect.

Surface	Where	Notes
Admin area	/admin	Site admins only. Seeding, moderation queue, sponsorships, recompute tools. Calls privileged backend routes using a shared secret.
Partner and group dashboards	/partners/dashboard, /groups/[slug]/organizer, stats pages	Role-gated views for business admins and group admins.

3.2 Main website sections

Route area	Purpose
/campaigns/[slug]	Campaign map (MapLibre), contribution logging, feed, leaderboards, geographic stats layers
/groups, /groups/[slug]	Group directory, group pages, event creation, organizer dashboard, group stats
/cleanup-events/[id], /team-events/[id]	Event detail, RSVP, check-in, logging, galleries, stats and review pages
/leaderboard, /achievements, /users/[username], /profile	Rankings, badges, public and own profiles
/partners, /partners/[slug], /partners/apply	Partner businesses, offers, redemption, applications
/impact, /sponsorships, /billing	Payments v2: impact reporting, sponsorships, Plus subscription (admin preview only for now)
/login, /signup, /settings/*, /legal/*	Auth (email, Google, Apple on iOS), account and profile settings, legal
/how-points-work, /routes	Scoring explainer, route tracking

3.3 Roles

- **Visitor**: browses public pages and maps.
- **Regular user**: logs contributions, joins groups, RSVPs, redeems offers.
- **Group admin / event organizer**: creates events, logs totals, manages members.
- **Partner business admin**: manages listings and offers, validates redemptions.
- **Site admin**: moderation, seeding, settings, roles. Roles are enforced by RLS for direct calls and by backend checks for API calls (see section 11).

4. Component inventory

Component	Technology	Hosted on	Responsibility
Web frontend	Next.js 16.2 (App Router), React 19, TypeScript, Tailwind 4, MapLibre GL, Recharts, Framer Motion, GSAP, h3-js	Vercel	All UI, server rendering, session handling (src/proxy.ts), admin proxy routes (app/api/*)
Mobile shells	Capacitor 8 with Camera, Geolocation, Push, Share, Browser, Filesystem, Social Login, FCM, plus a custom background-geolocation plugin	Google Play, App Store	Native wrapper that loads the production website. Adds push, background GPS, native Google and Apple sign-in
API	FastAPI 0.115, Python 3.12, SQLAlchemy 2 async + asyncpg, GeoAlchemy2, Shapely, H3, Pydantic 2	Railway (Docker, health check /health)	Scoring, geo assignment, cleanup events, groups, payments, sponsorships, tiles (MVT), uploads, email, seeding

Component	Technology	Hosted on	Responsibility
Cron workers	Python scripts in backend/scripts/, same Docker image	Railway (4 cron services)	Territory decay (every 6h), event expiry (every 30 min), organizer reminders (12:00 daily), organizer follow-ups (12:30 daily)
Database	Supabase Postgres + PostGIS, 117 migrations on master, ~150 RLS policies, ~37 SQL functions	Supabase (Pro plan)	System of record for everything
Auth	Supabase Auth (email, Google OAuth, Apple)	Supabase	Users, sessions, JWTs
Realtime	Supabase Realtime channels	Supabase	Live map and feed updates
Push sender	Supabase Edge Function send-push (Deno, TypeScript)	Supabase	Triggered by inserts into user_notifications via a DB trigger; sends through FCM v1
File storage	Cloudflare R2 (S3 API, boto3)	Cloudflare	Contribution photos, avatars, logos, event galleries
Infrastructure as code	.railway/railway.ts	Railway	Declares the API service and the 4 cron services (applied manually with railway config apply)
CI	GitHub Actions: supabase-keepalive.yml	GitHub	Pings Supabase twice weekly. No test or build CI exists

5. Frontend in detail

- **Two data paths.** About 225 direct Supabase queries (`.from("table")`) across roughly 69 files handle auth, reads, and simple writes under RLS. About 150 call sites hit FastAPI with `fetch(NEXT_PUBLIC_FASTAPI_URL/api/...)` for logic-heavy work.
- **Supabase clients** live in `src/lib/supabase/` (browser, server, proxy). Next.js 16 uses `src/proxy.ts` instead of middleware.ts to refresh the session cookie.
- **Server routes** under `src/app/api/admin/*` verify the caller is a site admin, then call FastAPI's admin-prod routes with the shared `ADMIN_API_SECRET` header.
- **Map stack.** MapLibre GL renders base tiles from MapTiler and vector tiles for campaign layers from FastAPI's MVT endpoint (`/api/tiles/{campaign}/{z}/{x}/{y}.mvt`). The tile endpoint is why the backend is always-on rather than scale-to-zero.
- **Native bridge.** `NativeAppBridge.tsx` handles deep links, push permission and token registration, splash screen hide, and OAuth return. A `FrontlineNativeApp` user-agent suffix lets the server tell the app from mobile web.
- **Observability.** Sentry for client, server, and edge.

6. Backend in detail

FastAPI app in `backend/app`, organized as `api/routes` (HTTP), `services` (business logic), `db` (async engine), and `core/config.py` (pydantic settings from env).

Route module	Domain
contributions, problem_reports, decay, events	Core gameplay: submit contributions, point-in-polygon geo assignment, territory claims, hotspot events, decay and expiry

Route module	Domain
cleanup_events, team_events, groups	Hosted events, RSVP and check-in, organizer logging, group admin
leaderboard, geo_units, tiles, campaign_dashboard, users	Rankings, geographic units and stats, MVT vector tiles, dashboards
partners, sponsorships, payments	Partner offers and redemption, sponsorships, Stripe checkout and webhooks
upload, device_tokens, account, content_flags	R2 presigned uploads, push token registration, account deletion and export, moderation
admin, admin_prod, admin_roles	Seeding and wipe (non-prod only), privileged prod actions (shared secret), role management

Key services: `contribution_scoring`, `game_settings` (admin-tunable values), `geo`, `impact`, `payments`, `sponsorships`, `email` (Resend), `admin_roles`, and a `seeders` registry used by `POST /admin/seed`. Reports use WeasyPrint, openpyxl, and matplotlib.

7. Database (Supabase Postgres)

- PostGIS powers point-in-polygon assignment (zip, neighborhood, borough), proximity checks, and tile generation.
- Migrations are plain SQL in `supabase/migrations/` (001 to 117 on master, 109 to 120 on the payments branch per the deploy plan). They are append-only and applied to prod manually by the owner.
- RLS policies guard every table the frontend touches directly. FastAPI connects with a direct SQL URL and is not subject to RLS.
- Admin-tunable values live in a `game_settings` table (point values, caps, feature flags such as payments_enabled and payments_admin_preview).
- Database triggers create profiles on signup and fan out push notifications (Supabase Vault secrets plus net.http_post to the edge function).
- Open backlog: dev/public schema split, backup strategy, materialized views for performance (see dev-docs).

8. Third-party services

Service	Used for	Where configured
Supabase	Database, auth, realtime, edge functions, vault	Frontend and backend env, supabase/ folder
Vercel	Frontend hosting, preview deployments	Vercel project env
Railway	Backend API and cron hosting	Railway env + .railway/railway.ts
Cloudflare R2	Image storage	Backend env (R2_*)
Stripe	Frontline Plus subscriptions, checkout, webhooks, planned Connect payouts	Backend env (STRIPE_*); not live for users yet
Resend	Transactional email (reminders, notices)	Backend env (RESEND_API_KEY)
Firebase (project frontline-498904)	Cloud Messaging for Android and iOS push (APNs key uploaded)	Edge function secret, google-services.json, GoogleService-Info.plist
MapTiler	Base map tiles and styles (high usage; reduction scoped)	NEXT_PUBLIC_MAPTILER_KEY
Sentry	Error monitoring for frontend and backend	SENTRY_DSN, sentry.*.config.ts

Service	Used for	Where configured
Google Cloud, Apple Developer, Play Console	OAuth clients, signing, store listings	Personal developer accounts
GitHub	Source, auto-deploy triggers, keepalive workflow	tayloraj10/Frontline

9. Key flows end to end

9.1 Sign-in

Browser: Supabase Auth via supabase-js, session in cookies, refreshed by proxy.ts. Android: native Google Credential Manager through the social-login plugin. iOS: native Google and Apple sign-in. The ID token is exchanged for a Supabase session, so the wrapped website sees the same session it would in a browser.

9.2 Logging a contribution

The user places a pin (GPS or manual) and optionally a photo. The frontend asks FastAPI for a presigned R2 URL and uploads the photo directly to R2. It then POSTs the contribution to FastAPI, which runs a point-in-polygon geo lookup, proximity checks, scoring from game_settings, inserts rows, and updates leaderboards. Supabase Realtime pushes the change to open maps and feeds.

9.3 Push notification

A row is inserted into user_notifications. A DB trigger reads Vault secrets and calls the send-push edge function. It signs a Google service-account JWT, calls FCM v1 for each token in device_tokens, and prunes dead tokens. Tapping the notification deep-links through NativeAppBridge.

9.4 Scheduled jobs

Four Railway cron services run Python scripts that call the API's maintenance endpoints: decay, event expiry, organizer reminders, organizer follow-ups. Those endpoints have no credentials today (see section 11). A GitHub Action separately keeps Supabase active.

9.5 Payments (hidden from regular users)

The frontend starts Stripe Checkout through FastAPI. Stripe webhooks hit `/api/payments/webhooks/stripe`, are deduplicated in stripe_webhook_events, and update subscriptions and sponsorships. Visibility is controlled by `payments_enabled` (0 in prod) and `payments_admin_preview` (1, admins only). Notes say migrations 109 to 120 and the frontend are live in prod, with the backend deploy pending a Railway outage (verify).

10. Environments, deploy, and release

Environment	Details
Local	Supabase CLI local stack, uvicorn backend, next dev. Seeding goes through the Seeder registry and POST /admin/seed.
Production	Vercel (frontend), Railway (API + 4 crons), Supabase prod project, R2 bucket frontline-uploads. Prod migrations are applied by the owner only.
Staging	Scoped but not built (dev-docs/security-lockdown-and-staging-2026-10-17.md). Vercel previews exist per branch; CORS allows this team's preview URLs.

Deploy order for schema-affecting changes: migrations to prod, then backend, then frontend. **Web:** push to master; Vercel builds the frontend, and Railway builds only when backend/** changes. **Mobile:** app content updates with the web deploy. Native changes (plugins, permissions, icons) need a new signed build. Android builds an .aab with `npm run build:android` and uploads to Play Console. iOS archives in Xcode on a Mac after `npm run sync:ios-version`. Bundle all native-impacting changes into one release.

Single points of failure: the Android signing key (frontline-app-signing-key.jks) has no backup yet. Play App Signing was switched to a classical key after the hybrid post-quantum key broke Google sign-in.

11. Coupling, portability, and security posture

Layer	Coupling to Supabase	Move cost
Postgres + PostGIS	Low. Standard Postgres; FastAPI uses a plain DATABASE_URL.	Easy (dump and restore)
FastAPI	Low. One call to the Supabase Auth admin API (delete user).	Easy
R2, Stripe, Resend, Sentry	None	None
Auth	High. Frontend login flows, OAuth, session cookies, auth.users foreign keys.	Hard
RLS	High. About 150 policies, 57 use auth.uid().	Hard
Frontend direct DB calls	Highest. About 225 queries plus Realtime channels.	Largest job
Edge function, vault, net.http_post trigger	Medium. One function.	Medium

Security posture (as documented 2026-10-17). The FastAPI backend has no JWT verification on most routes. Identity parameters such as user_id and requesting_user_id are trusted from the client, and the backend bypasses RLS. The role-management router is mounted in prod and trusts requesting_user_id. Cron-invoked endpoints are public. A staged lockdown plan exists (verify JWTs locally, an AUTH_ENFORCEMENT off/log/enforce switch, an X-Cron-Secret header, a shared frontend API client) but is not started. The admin-prod routes are protected by a shared secret and are fine as is.

12. Open items and risks

- Backend auth lockdown and a real staging environment (highest-priority risk).
- Backend deploy for Payments v2 pending; Stripe account setup, Connect payouts, legal and tax review still ahead.
- Back up the Android signing key.
- Android and iOS store review states: verify current status.
- Supabase backup strategy, schema split, and DB performance (materialized views) are scoped but not done.
- MapTiler usage is heavy; a reduction plan is scoped.
- NYC borough stats layer is not functional in prod until borough data is loaded there.
- No automated test or build CI. Only the Supabase keepalive workflow exists.
- Duplicated cleanup event forms (CreateCleanupEventForm and the ContributionPanel modal) must be kept in sync by hand.
- More email flows planned: partner and organizer coordination, attendee reminder blast, day-before organizer reminder.

13. Environment variable map (names only, no values)

Where	Variables
Frontend (Vercel)	NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, NEXT_PUBLIC_FASTAPI_URL, NEXT_PUBLIC_MAPTILER_KEY, NEXT_PUBLIC_APP_STORE_URL, NEXT_PUBLIC_PLAY_STORE_URL, ADMIN_API_SECRET, Sentry vars

Where	Variables
Backend (Railway)	DATABASE_URL, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, R2_ACCOUNT_ID, R2_ACCESS_KEY_ID, R2_SECRET_ACCESS_KEY, R2_BUCKET_NAME, R2_PUBLIC_URL, ENVIRONMENT, SENTRY_DSN, ADMIN_API_SECRET, RESEND_API_KEY, FRONTEND_URL, CORS_ORIGINS, STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, STRIPE_PRICE_ID_INDIVIDUAL, STRIPE_PRICE_ID_GROUP
Supabase (edge function, vault)	FIREBASE_SERVICE_ACCOUNT_JSON, push_function_url and push_service_role_key vault secrets
GitHub Actions	SUPABASE_URL, SUPABASE_ANON_KEY

Related docs in dev-docs: campaign-app-scope, capacitor-scoping-2026-08-04, android-release-checklist, ios-release-checklist, push-notifications-prod-deploy-checklist, security-lockdown-and-staging-2026-10-17, payments-v2-deployment-plan-2026-10-20, master-backlog, todo. External: app-capability-doc.

14. Local development setup (new developer)

Goal: run the website, the FastAPI backend, and a full local Supabase (Postgres, Auth, Realtime, Studio) on one machine, with no connection to production. Commands assume a terminal at the repo root. Version numbers marked **verify** are reasonable minimums, not tested pins.

14.1 Prerequisites

- **Git** and access to the GitHub repo tayloraj10/Frontline.
- **Node.js 20 or newer** with npm (verify against the Next.js 16 requirement).
- **Python 3.12** (matches backend/Dockerfile).
- **Docker Desktop**, running. The Supabase CLI starts its services as containers.
- **Supabase CLI**, used as `npx supabase` so no global install is needed.
- Optional: Android Studio (Android app), a Mac with Xcode (iOS app), the Railway CLI (cron and infra changes only).
- Accounts you may need keys from: MapTiler (free key is enough for dev), Cloudflare R2 (only to test photo uploads), Resend (only to test real email), Sentry (optional). Ask the project owner for any shared dev credentials. **Never use production keys or the production database URL locally.**

14.2 Clone and install

```
git clone https://github.com/tayloraj10/Frontline.git frontline
cd frontline

# frontend
cd frontend && npm install && cd ..

# backend (virtual environment recommended)
cd backend
python -m venv .venv
.venv\Scripts\activate          # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
cd ..
```

14.3 Start local Supabase

```
npx supabase start
npx supabase status      # prints API URL, DB URL, anon key, service_role key
```

The first start downloads images and takes several minutes. It applies every file in `supabase/migrations/` in order and runs `supabase/seed.sql` if present. Default local ports (supabase/config.toml): API 54321, Postgres 54322, Studio 54323, Inbucket (catches auth emails) 54324. Open Studio at http://localhost:54323 to browse tables, and Inbucket to read signup and password-reset emails, since no real email is sent locally.

Do **not** run `supabase db reset` casually: it wipes local data, including anything seeded or pulled from a snapshot. Apply new migrations with `npx supabase migration up` instead.

14.4 Backend environment and run

Copy `backend/.env.example` to `backend/.env` and set the minimum for local use:

```
DATABASE_URL=postgresql+asyncpg://postgres:postgres@127.0.0.1:54322/postgres
SUPABASE_URL=http://127.0.0.1:54321
SUPABASE_SERVICE_ROLE_KEY=<service_role key from supabase status>
ENVIRONMENT=development
ADMIN_API_SECRET=<any long random string>
```

```
CORS_ORIGINS=http://localhost:3000
```

R2, Resend, Sentry, and Stripe variables can stay blank unless you are working on those features. `ENVIRONMENT=development` is required to mount the admin seeding router (it is hidden when the value is production).

```
cd backend
uvicorn app.main:app --reload --port 8000     # verify the entrypoint module
```

Check http://localhost:8000/health and the interactive docs at http://localhost:8000/docs.

14.5 Frontend environment and run

Create `frontend/.env.local`:

```
NEXT_PUBLIC_SUPABASE_URL=http://127.0.0.1:54321
NEXT_PUBLIC_SUPABASE_ANON_KEY=<anon key from supabase status>
NEXT_PUBLIC_FASTAPI_URL=http://localhost:8000
SUPABASE_SERVICE_ROLE_KEY=<service_role key>
ADMIN_API_SECRET=<same value as the backend>
NEXT_PUBLIC_MAPTILER_KEY=<your MapTiler key>

cd frontend
npm run dev
```

`npm run dev` runs `scripts/dev.mjs`, which detects your LAN IP and overrides the Supabase and FastAPI URLs to `http://[lan-ip]:54321` and `:8000` so a phone or emulator on the same network can reach them. The browser still works at http://localhost:3000. If your Wi-Fi changes, the script clears the `.next` cache and re-syncs Capacitor automatically. See `dev-docs/capacitor-testing-guide.md` for native testing. Because of the override, Supabase and the backend must be reachable on the LAN IP; check your firewall if the page loads but data does not.

14.6 Get some data to look at

- **Seeding (the only supported way).** All seeding goes through the Seeder registry via `POST /admin/seed` on the local backend, not ad hoc scripts. Call it from the interactive docs at /docs, or from the Admin page of the running app. Seeders create campaigns, geo units, sample users, and contributions.
- **Geographic boundaries** (NYC boroughs, neighborhoods, ZIPs, and so on) load from GeoJSON files in `backend/data/` through the admin geo-load routes. Needed for the Geographic Stats layers.
- **Optional prod snapshot.** `backend/scripts/pull_prod_snapshot.sh` copies production data into local Postgres using a connection string you pass in. It needs credentials the project owner controls and replaces local data. Use only with explicit permission, and never commit the URL.
- **Make yourself an admin.** Sign up in the local app, read the confirmation email in Inbucket (http://localhost:54324), then set your profile's admin flag in Studio (the profiles table; verify the column name in migrations).

14.7 Day-to-day commands

Task	Command
Frontend dev server	cd frontend; npm run dev
Frontend lint / production build	npm run lint; npm run build
Backend dev server	cd backend; uvicorn app.main:app --reload --port 8000
Start / stop local Supabase	npx supabase start; npx supabase stop
Create a migration	npx supabase migration new [name], edit the SQL file, then npx supabase migration up

Task	Command
Apply pending local migrations	npx supabase migration up
Sync Capacitor after native changes	cd frontend; npx cap sync android (or ios)

14.8 Project rules a new developer must know

- **Production is live.** Migrations are applied to prod only by the project owner. Never run `supabase link` or `db push` against prod yourself.
- Seed data only through the Seeder registry and `POST /admin/seed`.
- Keep the two cleanup event forms in sync: `CreateCleanupEventForm.tsx` and the modal inside `ContributionPanel.tsx`.
- Do not commit `.env` files. `backend/.env.prod` exists on the owner's machine and must never be committed or used by a dev server.
- After feature work, update `campaign-app-scope.md` and `external-docs/app-capability-doc.md`.
- Do not use em dashes in copy, comments, or docs (house style).
- Work on a branch and open a PR to master. Master auto-deploys to Vercel, and to Railway when backend files change.

14.9 Troubleshooting

Symptom	Likely cause and fix
supabase start fails or hangs	Docker not running, or a port (54321 to 54324) is in use. Start Docker, run npx supabase stop, retry.
Frontend loads but login or data fails	NEXT_PUBLIC_SUPABASE_ANON_KEY is wrong or stale. Re-copy from npx supabase status. Restart the dev server after changing env values.
Browser works but phone or emulator does not	Firewall blocks ports 3000, 8000, 54321 on the LAN, or the IP changed. Re-run npm run dev.
Backend 500 on every request	DATABASE_URL points at the wrong port (local Postgres is 54322, not 5432) or Supabase is not running.
CORS errors in the browser console	Add the exact frontend origin to CORS_ORIGINS in backend/.env and restart the backend.
Admin seed route returns 404	ENVIRONMENT is production, or the admin router is disabled. Set ENVIRONMENT=development.
Map is blank	NEXT_PUBLIC_MAPTILER_KEY missing or restricted to other domains.
Odd characters when printing DB text in a Windows terminal	Console display issue, not data corruption. Check bytes via hex before assuming corruption.